

INFLUXDB + GRAFANA

Paso a paso

ÍNDICE

- Instalación de programas necesarios para la ingesta de datos
- InfluxDB
- Grafana

INSTALACIÓN DE PROGRAMAS NECESARIOS PARA LA INGESTA DE DATOS - PYTHON

- En el terminal `□ sudo apt update`
- En la terminal escribir `□ sudo apt install python3.12-venv`

```
jam@jejegod:~$ sudo apt install python3.12-venv
```

- Nos pregunta de nuevo si queremos descargar X cosas `□ y □ Enter`
- Para crear un contenedor `□ python3 -m venv bda` (o el nombre que le queramos dar a ese contenedor) `jam@oblivion:~$ python3 -m venv prensa`

- Entramos en el contenedor `□ source bda/bin/activate`

```
jam@oblivion:~$ source prensa/bin/activate  
(prensa) jam@oblivion:~$
```

- Si lo hacemos bien, aparece `(bda)`, todo lo que instalemos de Python a partir de ahora se instalará en este contenedor digital.

INSTALACIÓN DE PROGRAMAS NECESARIOS PARA LA INGESTA DE DATOS - PYTHON

- Instalamos la librería pandas □ se utiliza para tratar las bases de datos y prepararlas para la ingesta de datos
- pip3 install pandas

```
jam@oblivion:~$ source prensa/bin/activate
(prensa) jam@oblivion:~$ pip3 install pandas
Collecting pandas
  Downloading pandas-1.5.1-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (12.1 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.1/12.1 MB 3.5 MB/s eta 0:00:00
Collecting numpy>=1.21.0
  Downloading numpy-1.23.4-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (17.1 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.4/17.1 MB 3.2 MB/s eta 0:00:06
```

- Instalamos jupyter: se trata de un notebook, con el que vamos a trabajar para utilizar los comandos de Python.
- pip install notebook

```
(dba) jam@jejegod:~$ pip install notebook
Collecting notebook
  Downloading notebook-7.2.2-py3-none-any.whl.metadata (10 kB)
Collecting jupyter-server<3,>=2.4.0 (from notebook)
  Downloading jupyter_server-2.14.2-py3-none-any.whl.metadata (8.4 kB)
Collecting jupyterlab-server<3,>=2.27.1 (from notebook)
  Downloading jupyterlab_server-2.27.3-py3-none-any.whl.metadata (5.9 kB)
Collecting jupyterlab<4.3,>=4.2.0 (from notebook)
```

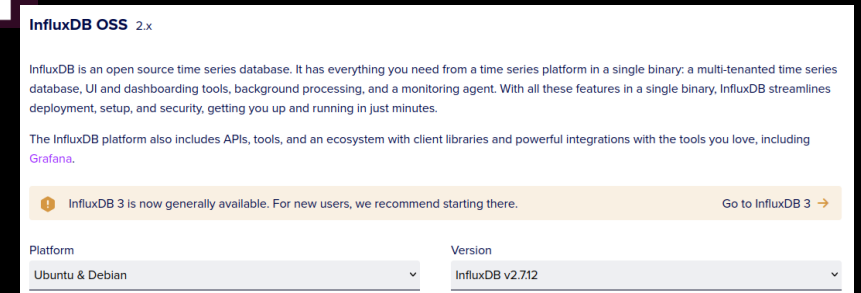
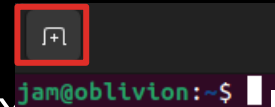
INSTALACIÓN DE PROGRAMAS NECESARIOS PARA LA INGESTA DE DATOS - PYTHON

- Instalamos la librería influxdb □ para poder acceder a los comandos relacionados con influxdb
- `pip3 install influxdb-client`

```
(prensa) jam@oblivion:~$ pip3 install influxdb-client
Collecting influxdb-client
  Downloading influxdb_client-1.34.0-py3-none-any.whl (707 kB)
    707.8/707.8 KB 3.2 MB
```

INSTALACIÓN DE PROGRAMAS NECESARIOS PARA LA INGESTA DE DATOS - INFLUXDATA

- Abrimos otra pantalla en la terminal
- A su vez vamos a Firefox y buscamos Influxdata
- <https://portal.influxdata.com/downloads/>
- Elegimos la versión y plataforma (2º página).
- Nos aparecen unas líneas de código.
- Copiamos la primera parte (Ctrl+C), vamos a la terminal y click con la rueda del ratón, o click derecho y pegar. ☐ Enter ☐ contraseña
- Copiamos la segunda parte
- Nos pide confirmar ☐ s ☐ enter



```
jam@oblivion:~$ # influxdb.key GPG Fingerprint: 05CE15085FC09D18E99EFB22684A14CF2582E0C5
wget -q https://repos.influxdata.com/influxdb.key
echo '23a1c8836f0afc5ed24e0486339d7cc8f6790b83886c4c96995b88a061c5bb5d influxdb.key' | sha256sum -c && c
at influxdb.key | gpg --dearmor | sudo tee /etc/apt/trusted.gpg.d/influxdb.gpg > /dev/null
echo 'deb [signed-by=/etc/apt/trusted.gpg.d/influxdb.gpg] https://repos.influxdata.com/debian stable mai
n' | sudo tee /etc/apt/sources.list.d/influxdata.list
jam@oblivion:~$ sudo apt-get update && sudo apt-get install influxdb2
```


INSTALACIÓN DE PROGRAMAS NECESARIOS PARA LA INGESTA DE DATOS - INFLUXDB

- Instalamos Telegraf ☐ mismo procedimiento, copiamos las líneas de código en la terminal

```
Version
Telegraf v1.24.3
Platform
Ubuntu & Debian
SHA256: ddcdf6b0b0412e50f914f10e13e2fc6931d329401b75620ca512539d720499
# influxdb.key GPG Fingerprint: 05CE15085FC09D18E99EFB22684A14CF2582E0C5
wget -q https://repos.influxdata.com/influxdb.key
echo '23a1c8836f0afc5ed24e0486339d7cc8f6790b83886c4c96995b88a061c5bb5d influxdb.key' | sha256sum
echo 'deb [signed-by=/etc/apt/trusted.gpg.d/influxdb.gpg] https://repos.influxdata.com/debian sti
sudo apt-get update && sudo apt-get install telegraf
```

INSTALACIÓN DE PROGRAMAS NECESARIOS PARA LA INGESTA DE DATOS - INFLUXDATA

- Para iniciar influxDB ☐ escribimos `influxd` en la terminal
- Vamos al navegador y escribimos `localhost:8086`
- Aparece una pantalla la primer vez de Get started

Setup Initial User

You will be able to create additional Users, Buckets and Organizations later

Username

Password Confirm Password

Initial Organization Name ⓘ
An organization is a workspace for a group of users.

Initial Bucket Name ⓘ
A bucket is where your time series data is stored with a retention policy.

somo

12345678

somorrostro

initial-bucket

INFLUXDB – PROCESAMIENTO DE DATOS

- Ya tenemos una cuenta en influxdb, vamos a ver cómo procesar los datos que tenemos que meterle.
- Para ello, entramos dentro de nuestro entorno virtual `source bda/bin/activate`
- Escribimos `jupyter notebook` se nos abre en el navegador una pestaña

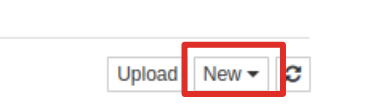
```
jam@oblivion:~$ source prensa/bin/activate
(prensa) jam@oblivion:~$ jupyter notebook
```

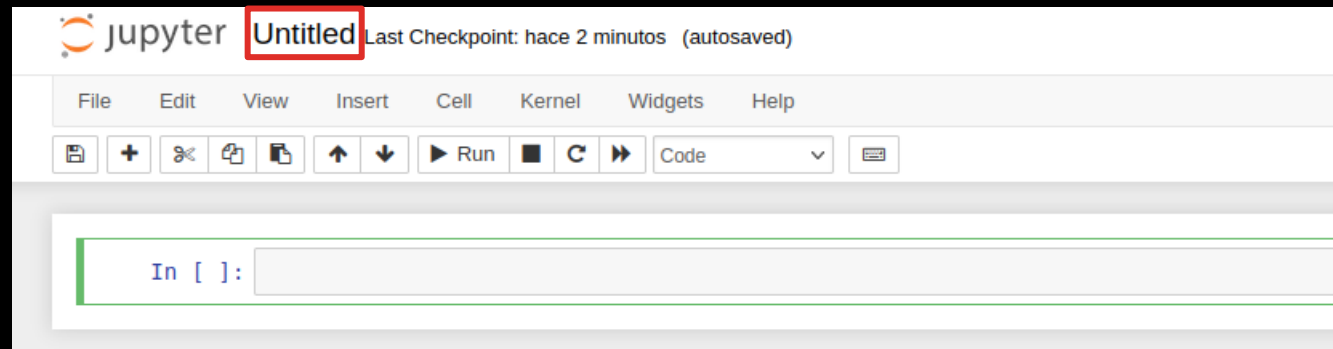


- Puede que necesitemos copiar un link que aparecerá en la terminal para poder abrir el notebook.
(<http://localhost:8888> /...)

Para poder hacer este paso, compartir con los alumnos algún csv, o utilizar Google trends para crear diferentes csvs. Poner el csv en la carpeta personal.

INFLUXDB – PROCESAMIENTO DE DATOS

- Creamos un nuevo notebook ☐ click en New ☐  se nos abre otra pestaña, donde iremos escribiendo el código para modificar la database que tenemos
- Cambiamos el nombre de este notebook ☐ Click en Untitled y ponemos el nombre que queramos.



INFLUXDB – PROCESAMIENTO DE DATOS

- Lo primero que hacemos es importar dos bibliotecas de python3.

```
import pandas as pd
import datetime
```

- Para correr el código, le damos a  , o le damos a Ctrl+Enter o Shift+Enter
- Metemos en una variable que llamamos “rawdata” el csv.
- Si queremos ver qué lleva el csv escribimos el nombre de esa variable.

```
rawdata = pd.read_csv('prensa.csv', sep = ',')
```

```
In [3]: rawdata
Out[3]:
```

	epoch	theta(rad)	ID(A)	IQ(A)	F(tn)
0	1666080599	-3.141108	0.538184	-4.305469	0.395879
1	1666080600	-3.141108	0.000000	-3.946680	0.395879
2	1666080601	-3.141108	0.000000	-3.946680	0.395879
3	1666080602	-3.141108	0.717578	-3.946680	0.395879
4	1666080603	-3.141108	0.538184	-3.587891	0.395879
...	...	...	...	...	...
34790	1666115389	3.426915	-0.896973	-0.538184	0.177314
34791	1666115390	3.426915	-0.896973	-0.538184	0.184161
34792	1666115391	3.426915	2.152735	-3.587891	0.190675
34793	1666115392	3.426915	3.408496	-4.484864	0.196436
34794	1666115393	3.426915	1.973340	-5.023047	0.201025

34795 rows × 5 columns

INFLUXDB – PROCESAMIENTO DE DATOS

- En el siguiente paso, convertimos la columna de “epoch” en una fecha con un formato más habitual.

```
In [4]: rawdata['epoch'] = rawdata['epoch'].apply(lambda x: datetime.datetime.fromtimestamp(x))
```

- Ahora queremos convertir epoch en el índice de nuestro csv, ya que influxdb necesita los csvs con ese formato.
- A la vez, borramos la columna ‘epoch’, porque la acabamos de poner como índice, y no la queremos repetida dos veces.

```
In [5]: rawdata
```

```
Out[5]:
```

	epoch	theta(rad)	ID(A)	IQ(A)	F(tn)
0	2022-10-18 10:09:59	-3.141108	0.538184	-4.305469	0.395879
1	2022-10-18 10:10:00	-3.141108	0.000000	-3.946680	0.395879
2	2022-10-18 10:10:01	-3.141108	0.000000	-3.946680	0.395879

```
In [6]: rawdata.set_index(rawdata['epoch'], inplace = True)
```

```
In [7]: del rawdata['epoch']
```

```
In [8]: rawdata
```

```
Out[8]:
```

	theta(rad)	ID(A)	IQ(A)	F(tn)
epoch				
2022-10-18 10:09:59	-3.141108	0.538184	-4.305469	0.395879
2022-10-18 10:10:00	-3.141108	0.000000	-3.946680	0.395879
2022-10-18 10:10:01	-3.141108	0.000000	-3.946680	0.395879

Con la nueva versión de Influxdb ha cambiado esta parte, ya no se puede acceder al token cuando queramos, solo en la creación, por lo que creamos un API Token nuevo.

INFLUXDB – SUBIDA DE DATOS

- Cargamos la librería de Influxdb

Subida de datos a InfluxDB

```
In [12]: from influxdb_client import InfluxDBClient
```

- Tenemos que cargar un token, personal para cada uno, para ello, vamos a la pestaña de influxdb (localhost:8086) ☐ click en el icono de la flecha ☐ opción de API Tokens
- En esta ventana, creamos un nuevo API Token ☐ Generate API Token ☐ All access API Token ☐ COPIAMOS EL TOKEN QUE NOS DA Y LO GUARDAMOS EN UN TXT
- En el jupyter notebook, guardamos este token en una variable my_token

```
In [13]: my_token="E9xb5S5n9Bc-7lBd96ow7nVqxp53s1zzvr3SNIc1ErFKk5R4rTDB6hBYH0uh_3cZdKcvDDuhA0PpSi0Tb16w==Gwp_Epltg=="
```

INFLUXDB – SUBIDA DE DATOS

- Cargamos el cliente de Influxdb

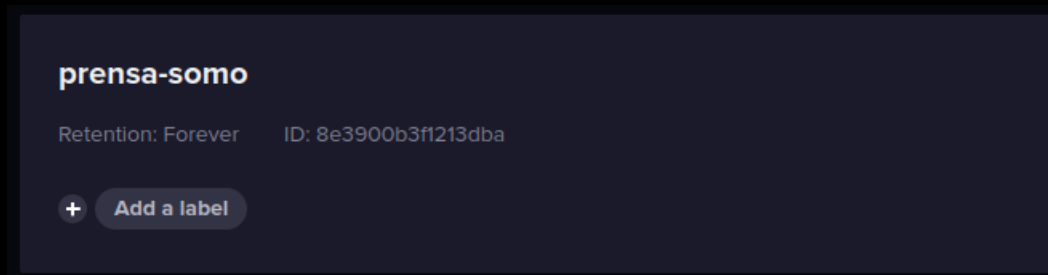
```
In [14]: client = InfluxDBClient(url="http://localhost:8086", token=my_token, org="somorrostro")
```

- Escribimos en ese cliente

```
In [15]: write_client = client.write_api()
```

- Antes de continuar vamos a la pestaña de Influxdb, y vamos a crear el bucket que vamos a utilizar para contener los datos  volvemos al icono de la flecha  Bucket 

- Click en Create Bucket  le ponemos nombre (**datos-bucket**)

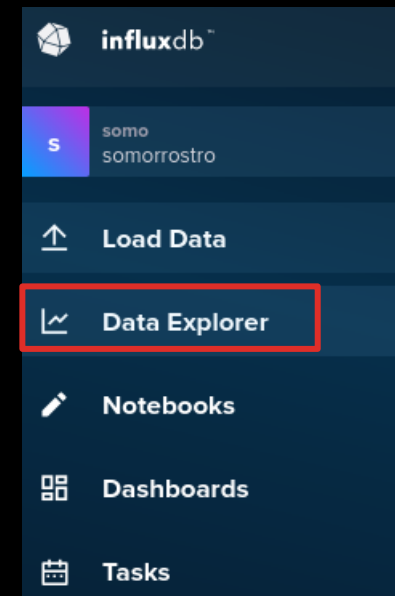


INFLUXDB – SUBIDA DE DATOS

- Volvemos a la pestaña de jupyter, y metemos los datos que hemos procesado en el bucket que hemos creado. También tenemos que crear el nombre del measurement, que será donde nos aparezcan los datos que estamos importando.

```
In [13]: write_client.write("prensa-somo", record=dframe, data_frame_measurement_name="datos-measure")
```

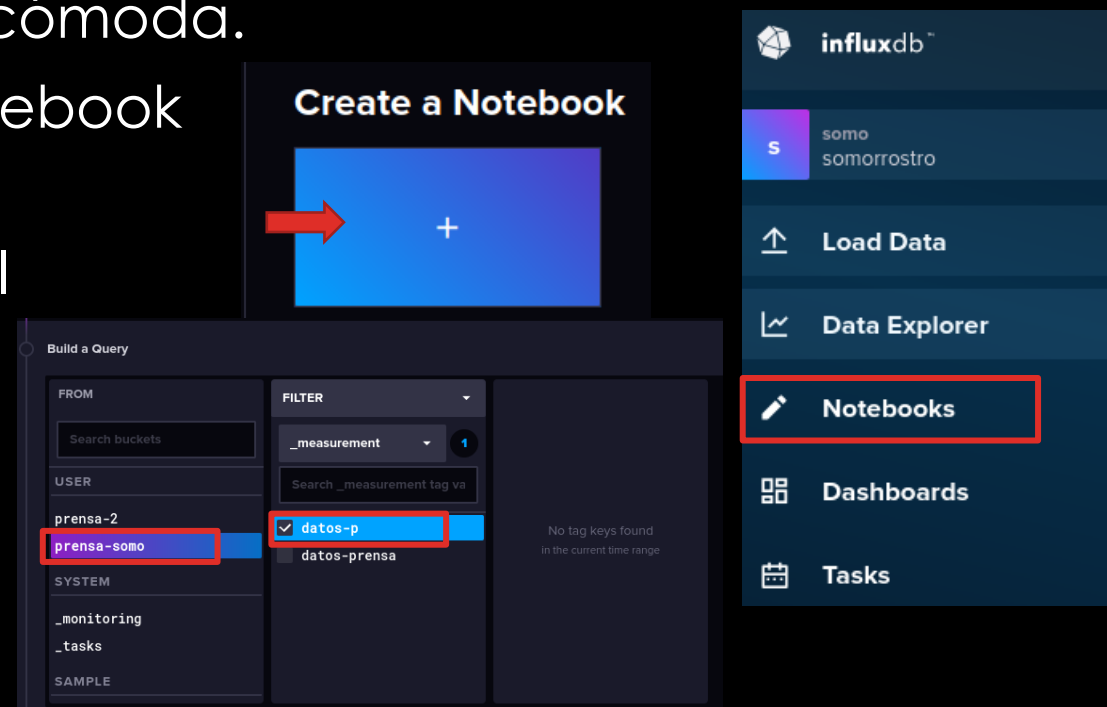
- Dentro del mismo bucket podemos tener distintos measurement, para almacenar distintos datos de manera ordenada.
- Si vamos a la opción Data Explorer, si todo ha ido correctamente, nos aparecerá ahí el bucket, con el measurement y con los datos.



** Puede ser que no nos aparezca el measurement en este paso ☐ cambiar el rango de tiempo como se explica en la siguiente diapositiva.

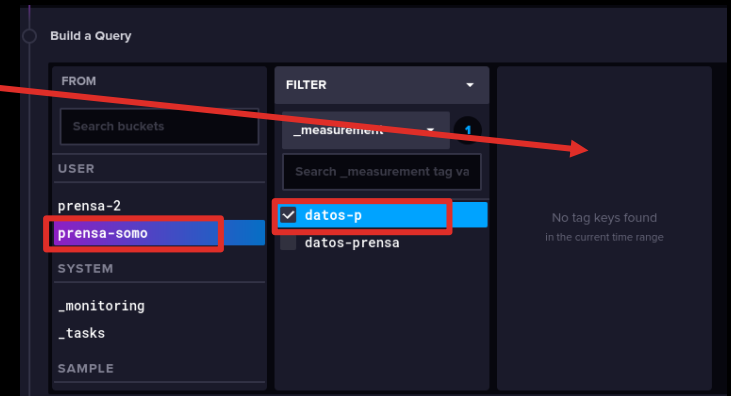
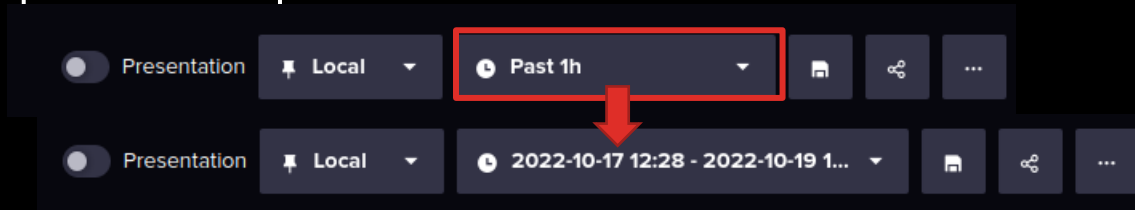
INFLUXDB – MOSTRAR DATOS

- Para mostrar estos datos ☐ podemos hacerlo directamente en la pestaña de data explorer.
- Otra manera es mediante un notebook, lo que nos permitirá más adelante volver a esos datos de una manera más cómoda.
- En la pestaña de Notebooks ☐ New Notebook
- Entraremos en una nueva ventana. En la parte superior cambiamos el nombre del notebook. (p. ej. Datos prensa).
- Elegimos de qué Bucket y measurement queremos sacar los datos.

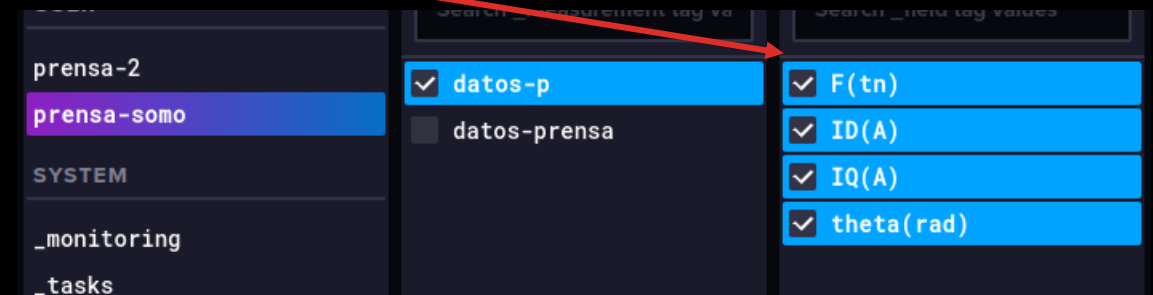


INFLUXDB – MOSTRAR DATOS

- Puede ocurrir que no aparezcan datos.
- Para que aparezcan, cambiamos el rango de tiempo, y ponemos uno tiempo que nos incluya ese tiempo que nos aparecía en el csv o rawdata.

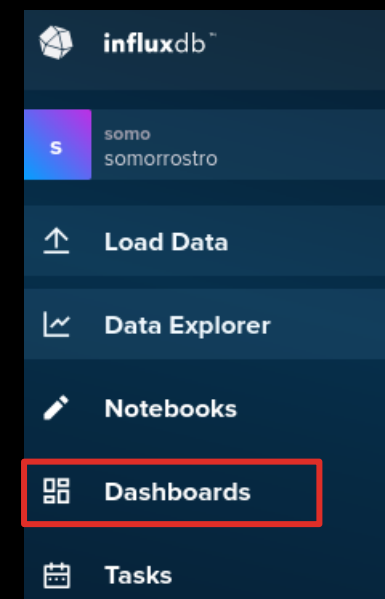


- Ahora nos aparecerán las variables (field_) que podemos mostrar.
- Para que nos las grafique ☐ RUN



INFLUXDB – MOSTRAR DATOS

- En la parte inferior aparecerá el gráfico de las variables seleccionadas.
- En el apartado Dashboard podemos crear también gráficas de rápido acceso, de una manera muy similar a la explicada para el notebook.



INFLUXDB – MOSTRAR DATOS

- Posibles prácticas:
 - Google trends
 - App de móvil □ Phyphox
 - Ourworldindata.org

GRAFANA – MOSTRAR DATOS

- Grafana va a ser el programa que se a utilizar para mostrar los datos.
- Para instalarlo ☐ abrimos el terminal
- A su vez vamos a la página web de grafana

<https://grafana.com/docs/grafana/latest/setup-grafana/installation/debian/>

- Seguimos los pasos que nos marcan, escribiendo el código en la terminal, hasta el paso 6 (no necesitamos Grafana Enterprise).
- **NO INSTALAR** el repositorio 4 (beta releases)

5 Run the following command to update the list of available packages:

```
bash
# Updates the list of available packages
sudo apt-get update
```

6 To install Grafana OSS, run the following command:

```
bash
# Installs the latest OSS release:
sudo apt-get install grafana
```

1 Install the prerequisite packages:

```
bash
sudo apt-get install -y apt-transport-https software-properties-common wget
```

2 Import the GPG key:

```
bash
sudo mkdir -p /etc/apt/keyrings/
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg
```

3 To add a repository for stable releases, run the following command:

```
bash
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo tee -a /etc/apt/sources.list.d/grafana.list
```

4 To add a repository for beta releases, run the following command:

```
bash
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com beta main" | sudo tee -a /etc/apt/sources.list.d/grafana.list
```


GRAFANA – MOSTRAR DATOS

- Los comandos para iniciar, parar o ver si funciona Grafana son:
 - Iniciar `sudo systemctl start grafana-server`
 - Parar `sudo systemctl stop grafana-server`
 - Funciona? `sudo systemctl status grafana-server`
***Para esto también podemos usar el comando `netstat -nltp` (si no lo tenemos instalado, nos dirá cómo hacerlo)
- *La 1ª vez que lancemos grafana, usaremos este comando.

1 To start the service, run the following commands:

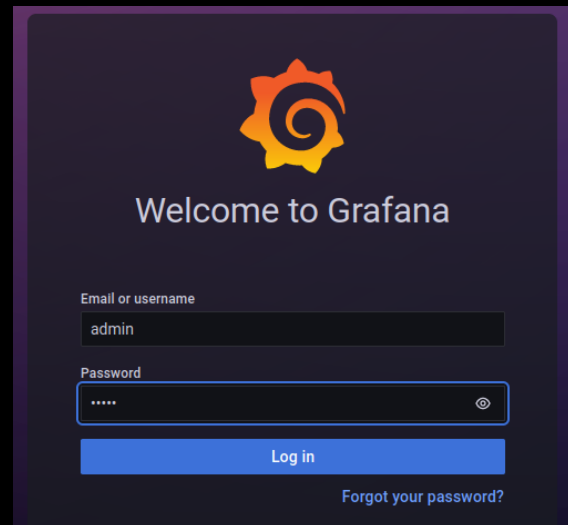
Bash

Copy

```
sudo systemctl daemon-reload
sudo systemctl start grafana-server
```

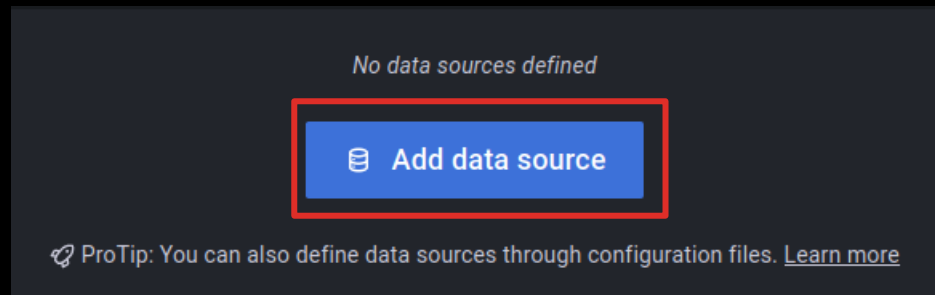
GRAFANA – MOSTRAR DATOS

- Una vez hemos iniciado grafana en el terminal, vamos a Firefox.
- <http://localhost:3000>
- User y PW: admin / admin, cuando nos diga para cambiar contraseña ☐ admin



GRAFANA – MOSTRAR DATOS

- Vamos a decirle qué tipo de datos tiene que leer.
- Abajo a la izquierda  Configuración  Data Sources  Add Data Sources



- Seleccionamos InfluxDB  se abre una ventana de configuración

GRAFANA – MOSTRAR DATOS

- Name: ds-influx
- Query Language: Flux
- HTTP: Url ☐ <http://localhost:8086>
- Basic Auth Details
 - User: somo (user de InfluxDB)
 - PW: 12345678 (la de InfluxDB)
- InfluxDB
 - Organization: somorrostro
 - Token: copiar el token de Influx
 - Default-Bucket: el bucket de influx

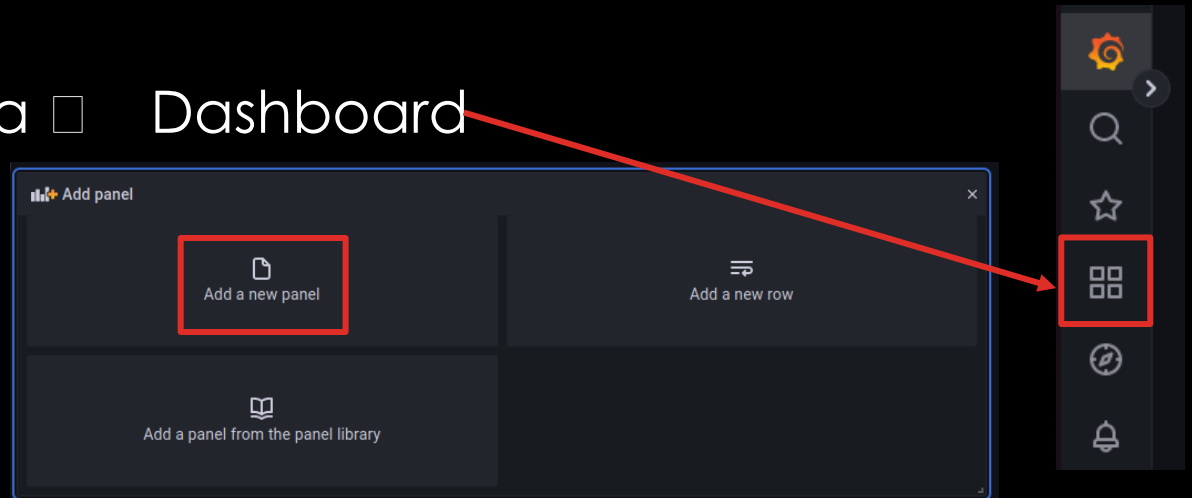
The screenshot shows the Grafana configuration interface for a new data source. The 'Name' field is set to 'ds-influx' and the 'Default' toggle is turned on. The 'Query Language' is set to 'Flux'. The 'HTTP' section shows the 'URL' as 'http://localhost:8086'. The 'Basic Auth Details' section shows the 'User' as 'somo' and the 'Password' as '12345678'. The 'InfluxDB Details' section shows the 'Organization' as 'somorostro', the 'Token' as '12345678', and the 'Default Bucket' as 'prensa-somo'. The 'Min time interval' is set to '10s' and the 'Max series' is set to '1000'.

Name	ds-influx	Default	☒
Query Language			
Flux			
HTTP			
URL	http://localhost:8086		
Allowed cookies	New tag (enter key to add) Add		
Timeout	Timeout in seconds		
Basic Auth Details			
User	somo		
Password	12345678		
InfluxDB Details			
Organization	somorostro		
Token	12345678		
Default Bucket	prensa-somo		
Min time interval	10s		
Max series	1000		

Quando
tengamos los
datos
introducidos ☐
Save and test

GRAFANA – MOSTRAR DATOS

- En el menú de arriba a la izquierda  Dashboard
- New  New Dashboard
- Add new panel



- Se abre una ventana donde elegiremos que info vamos a traer desde Influxdb, podemos cambiar el nombre del panel, el tiempo de lectura, etc.
- Para meter los datos, volvemos a InfluxDB.

GRAFANA – MOSTRAR DATOS

- Vamos al notebook que tenemos creado en InfluxDB
- Cogemos los datos que queremos llevar a Grafana, por ejemplo, la Fuerza.
- En los tres puntos de arriba a la derecha ☰ Convert to Flux
- Nos cambia la ventana y aparece código, lo copiamos.

The image shows the 'Build a Query' interface in Grafana. On the left, a code editor displays a Flux query:

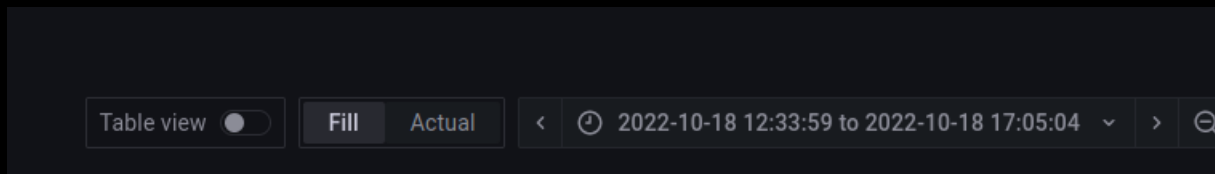
```
1 from(bucket: "prensa-somo")
2   |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
3   |> filter(fn: (r) => r["_measurement"] == "datos-p")
4   |> filter(fn: (r) => r["_field"] == "F(tn)")
```

A red arrow points from the third bullet point of the list to the top right corner of the interface. In this corner, there is a three-dot menu icon (☰). A red box highlights this icon, and another red box highlights the 'Convert to > Flux' option in the dropdown menu that appears.

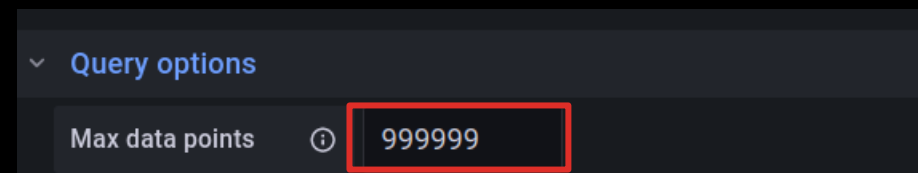
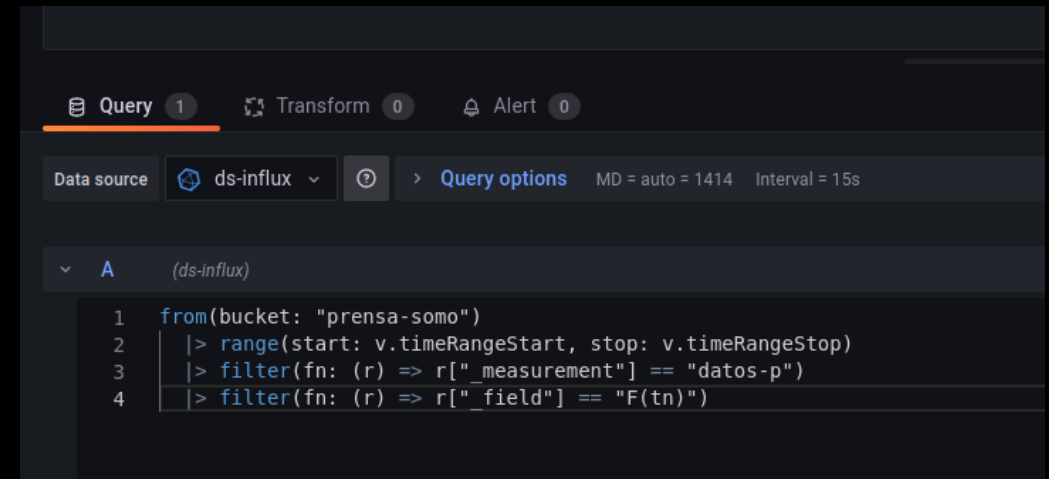
The main interface shows the 'FROM' section with 'prensa-somo' selected, and two 'FILTER' sections. The first filter is '_measurement' with 'datos-p' selected. The second filter is '_field' with 'F(tn)' selected. The right panel shows a list of available fields: 'ID(A)', 'IQ(A)', and 'theta(rad)'. The 'No tag keys found in the current time range' message is displayed in the center.

GRAFANA – MOSTRAR DATOS

- Volvemos a Grafana.
- Copiamos el código en la parte inferior
- Seleccionamos el tiempo de los datos



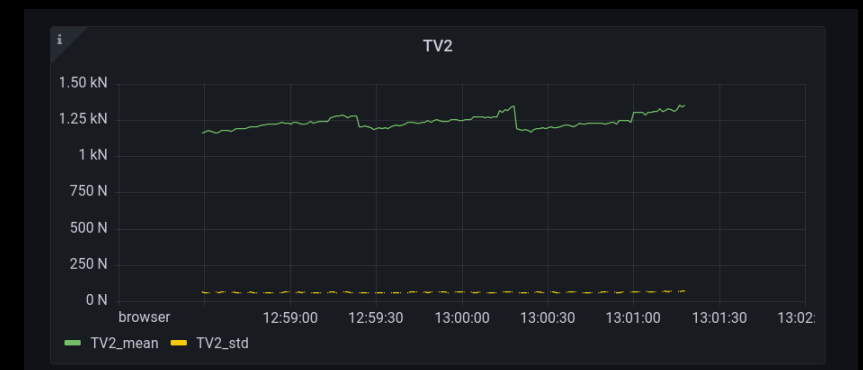
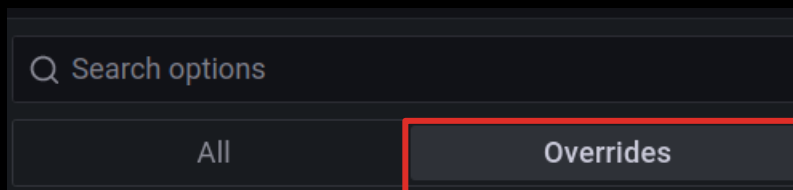
- Apply
- Podemos mirar la grafica clickando en ella y arrastrando.
- A la izquierda podemos cambiar distintos parámetros de las gráficas.
- Si no nos muestra todos los puntos ☐



GRAFANA – MOSTRAR DATOS

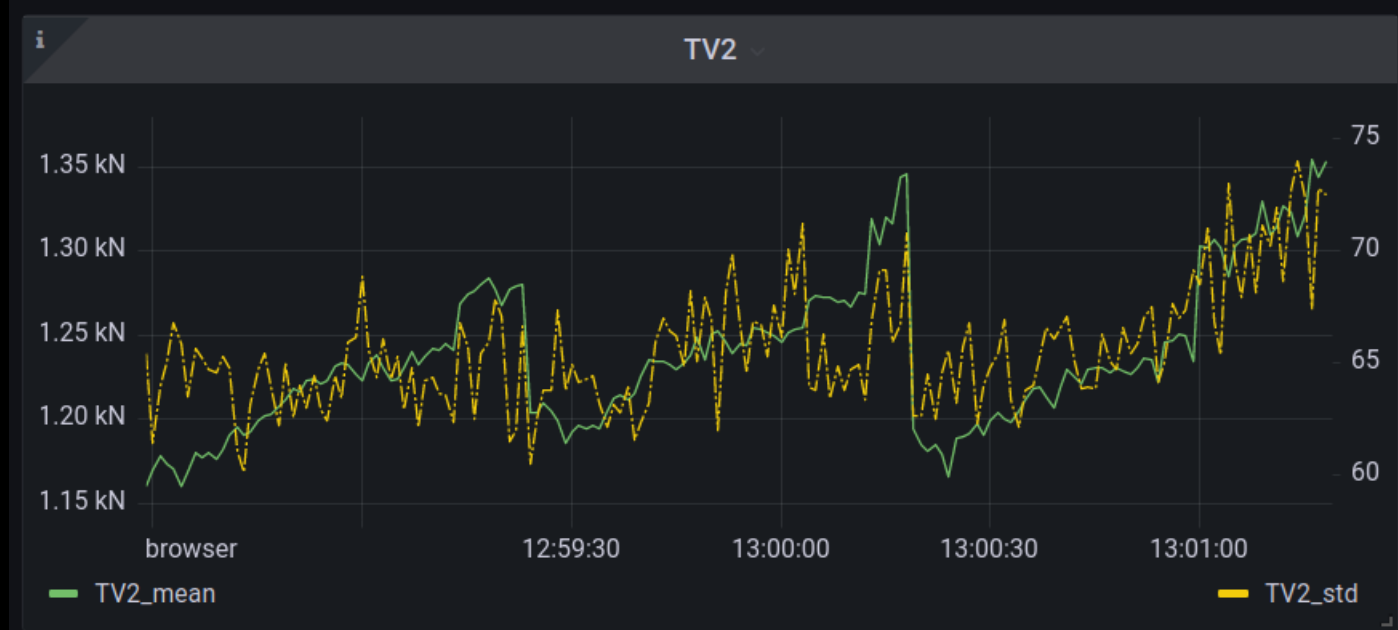
- Por ejemplo, si queremos mostrar en la misma gráfica dos datos, como pueden ser la media de la fuerza en el eje Z (TV2) y la desviación estándar de esa fuerza (TV2_std).
- Seguimos el procedimiento mostrado en las diapositivas anteriores, seleccionando en InfluxDB las dos variables.
- En grafana, nos muestra los datos juntos, a pesar de que las unidades no son las mismas para ambas variables.

Para poder trabajar con cada gráfica por separado, debemos elegir la opción ☐ Overrides



GRAFANA – MOSTRAR DATOS

- Esta opción nos va a permitir poner cada variable en una de los ejes, así como modificar cada una de las variables.
- Conseguiremos que la gráfica se muestre de la manera que buscamos.



Override 2

Fields with name
TV50_std

Axis > Placement
Auto Left **Right** Hidden

Axis > Label
Desviación estandar

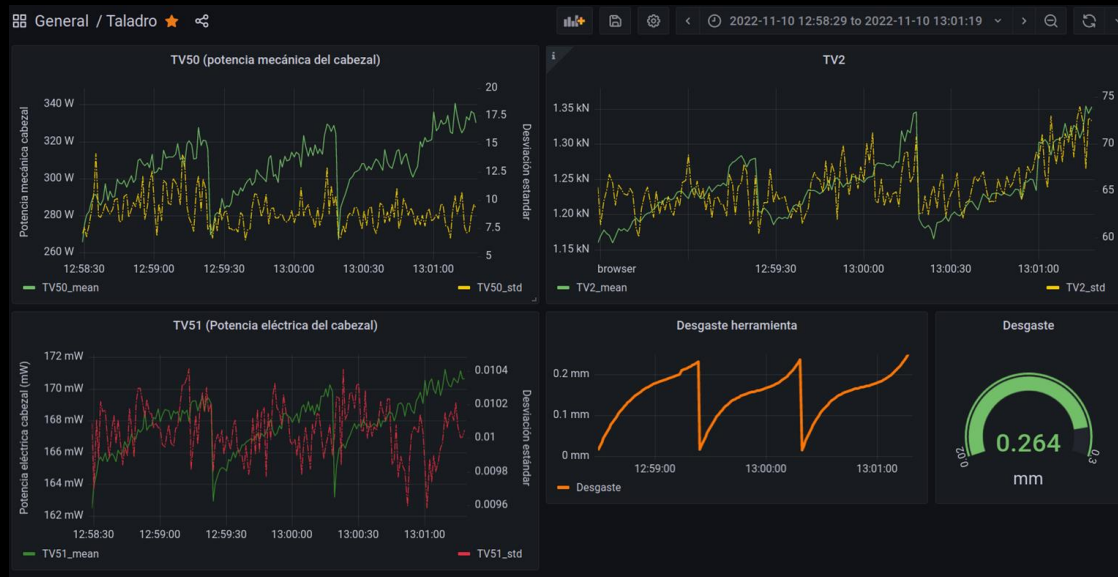
Graph styles > Line style
Solid **Dash** Dots 10, 2, 2, 2

Axis > Soft min
5

Axis > Soft max
20

GRAFANA – MOSTRAR DATOS

- En la parte superior, en Time series, podemos seleccionar otro tipo de gráfica, según la que nos interese mostrar.
- En un mismo Dashboard podemos tener múltiples paneles.



Q Search for...

Visualizations Suggestions Library panels

Time series
Time based line, area and bar charts

Bar chart
Categorical charts with group support **Beta**

12.4 Stat
Big stat values & sparklines

Gauge
Standard gauge visualization

Bar gauge
Horizontal and vertical gauges

Table
Supports many column styles

Pie chart
The new core pie chart visualization

MOSTRAR DATOS EN TIEMPO REAL (STREAMING) CON PYTHON

- Para mostrar datos en grafana en tiempo real, necesitamos también cambiar la manera de introducir los datos en InfluxDB, ya que ahora no tendremos un CSV “cerrado”, si no que los datos se irán actualizando en cada momento.
- Vamos a hacer el **primer ejemplo** con un archivo que crea ruido, pero la forma de trabajar será muy similar si tenemos un sensor, un robot o cualquier elemento que nos de información cada x tiempo.
- Utilizamos de muestra el archivo: “prensa-to-influx” como ejemplo.
- Puede ser que tengamos un archivo histórico inicial, la forma de cargar esa info es igual que antes, tanto en grafana como en InfluxDB.

MOSTRAR DATOS EN TIEMPO REAL (STREAMING)

- Para cargar archivos en streaming:
- Importamos las siguiente funciones, y definimos las variables mu y sigma.

```
In [33]: import time

import numpy as np
mu, sigma = 0, 0.1
```

- Al ejecutar la siguiente parte del código, con los últimos valores de las variables, les mete ruido para crear una variación
- A su vez, escribe los datos en un dataframe, para después concatenarlos a los datos que teníamos antes.
- Finalmente, los sube a influxdb.

```
for b in range(1,100):

    a = int(time.time())

    noise = np.random.normal(mu, sigma,1)
    clean_signal = rawdata['theta(rad)'].iloc[-1]
    signalA = clean_signal + noise

    noise = np.random.normal(mu, sigma,1)
    clean_signal = rawdata['ID(A)'].iloc[-1]
    signalB = clean_signal + noise

    noise = np.random.normal(mu, sigma,1)
    clean_signal = rawdata['F(tn)'].iloc[-1]
    signalC = clean_signal + noise

    noise = np.random.normal(mu, sigma,1)
    clean_signal = rawdata['IQ(A)'].iloc[-1]
    signalD = clean_signal + noise

    d = {'epoch':datetime.datetime.fromtimestamp(a), 'theta(rad)':signalA[0], 'ID(A)':signalB[0], 'IQ(A)':signalD[0], 'F(t)':signalC[0]}
    df2=pd.DataFrame(d,index=[0])
    df2.set_index(df2['epoch'], inplace = True)
    del df2['epoch']
    time.sleep(0.25)
#print(df2)
rawdata =pd.concat([rawdata,df2],axis=0)
write_client.write("20221103-prensa", record=rawdata, data_frame_measurement_name="datos-prensa")
```


MOSTRAR DATOS EN TIEMPO REAL (STREAMING)

- Vamos a nuestro panel de Graphana, con los datos que teníamos de antes.
- Cambiamos el tiempo del Time Range, según el tiempo que necesitamos mostrar, en este caso, el tiempo que crea cada vez que crea el ruido es el actual.
- Por cuestiones de Grafana e Influx, para que nos muestre el tiempo actual en Grafana, tenemos que poner `now+1h`



Absolute time range

From

now

To

now+1h

Apply time range

MOSTRAR DATOS EN TIEMPO REAL (STREAMING)

- En la esquina superior ☐  5s  Para que las gráficas se actualicen cada 5s
- Iremos viendo, cada vez que ejecutemos el comando de generar ruido, cómo se van actualizando las gráficas.

Para que las gráficas se actualicen cada



MOSTRAR DATOS EN TIEMPO REAL (STREAMING)

- Segundo ejemplo: taladro donde se han medido distintas variables para ver las más influyentes, se ha entrenado un modelo con algunos de los puntos medidos (80%), con otros se comprueba el modelo (20%). Finalmente con los datos de esas variables, se grafica el desgaste y otras variables.
- Para este documento voy a centrarme en la parte final, donde se meten ciertos puntos en Influx, para ver otra manera de hacerlo.
- El documento de referencia es MMSE_01 (Carpeta somo_taladro)

MOSTRAR DATOS EN TIEMPO REAL (STREAMING)

- Importamos los siguientes comandos

```
import influxdb_client
from influxdb_client import InfluxDBClient
import time
```

- Definimos el token, la organización, etc, el cliente...

```
my_token="D3_b0SN9Lv5bq8_-IPFwj_r_sDMbZEJQujnhSPucmM_R-9R70B_d4L8HzYzrC9tJX7eNZSg7P6hiI0GGsXGLw=="
org='somorrostro'
client = InfluxDBClient(url="http://localhost:8086", token=my_token, org=org)
write_client = client.write_api()
```

MOSTRAR DATOS EN TIEMPO REAL (STREAMING)

- Lanzamos el siguiente bloque

```
for i in range(0, len(X_train)):
    data_pred=X_train.iloc[[i],:]
    print(data_pred)
    prediction = rf.predict(data_pred)

    p = influxdb_client.Point("desgaste").tag("hta1", "hole").field("desgaste", float(prediction)).\
        field("TV50", float(data_pred['TV50_mean'])).\
        field("TV2", float(data_pred['TV2_mean'])).\
        field("TV51", float(data_pred['TV51_mean'])).\
        field("TV2_std", float(data_pred['TV2_std'])).\
        field("TV50_std", float(data_pred['TV50_std'])).\
        field("TV51_std", float(data_pred['TV51_std']))

    write_client.write("taladro", org, record=p)
    time.sleep(5)
```

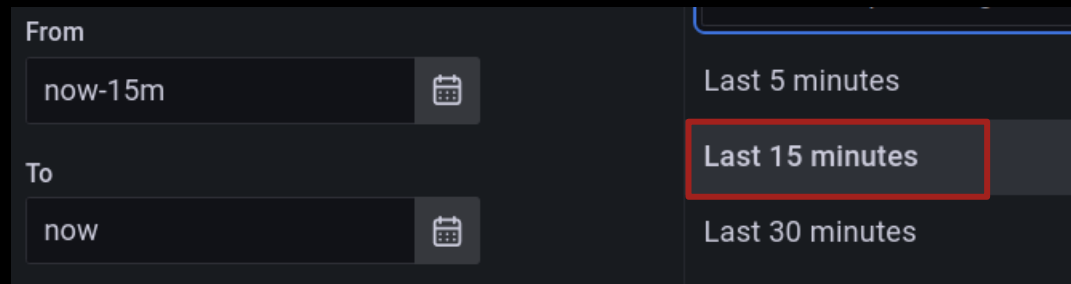


	TV50_mean	TV50_std	TV51_mean	TV51_std	TV2_mean	TV2_std	SREAL_mean	FREAL_mean	diametro	f	Vc
0	265.94	7.0692	0.16250	0.010092	1160.0	65.411	597.13	59.709	8	0.099993	15.007514
1	275.41	7.6432	0.16395	0.009703	1169.9	61.450	597.14	59.711	8	0.099995	15.007765
3	282.65	7.8970	0.16550	0.009883	1173.5	65.096	597.12	59.709	8	0.099995	15.007262
4	287.14	8.9592	0.16574	0.010221	1170.6	66.825	597.13	59.711	8	0.099997	15.007514
5	291.24	10.7480	0.16548	0.010160	1159.8	65.851	597.12	59.711	8	0.099998	15.007262
...	...	...	...	...	...	...	...	...	...	...	...
164	327.65	7.6833	0.17069	0.010107	1323.3	72.762	597.13	59.711	8	0.099997	15.007514
165	333.71	7.1272	0.17054	0.010210	1309.2	74.054	597.14	59.709	8	0.099992	15.007765
166	331.80	7.2731	0.17039	0.010072	1322.0	72.299	597.14	59.709	8	0.099992	15.007765
167	336.44	8.8544	0.17115	0.010006	1355.0	67.481	597.14	59.709	8	0.099992	15.007765
169	330.32	9.3487	0.17065	0.010050	1353.3	72.503	597.14	59.711	8	0.099995	15.007765

- Para el dataframe de X_train, para cada fila, desde la 0 hasta la última, hace una predicción del valor del desgaste.
- Se carga en influxdb, en el Bucket de "taladro", en el measure "desgaste", las distintas variables que tenemos y la única variable calculada, el desgaste.
- Se espera 5 segundos y vuelve a hacerse la operación para la siguiente fila.

MOSTRAR DATOS EN TIEMPO REAL (STREAMING)

- Vamos a Influx y hacemos el proceso de coger los datos y llevarlos a grafana.
- En Grafana, preparamos las distintas gráficas y señalamos que queremos tener los datos desde hace 15 minutos (por ejemplo) hasta ahora.

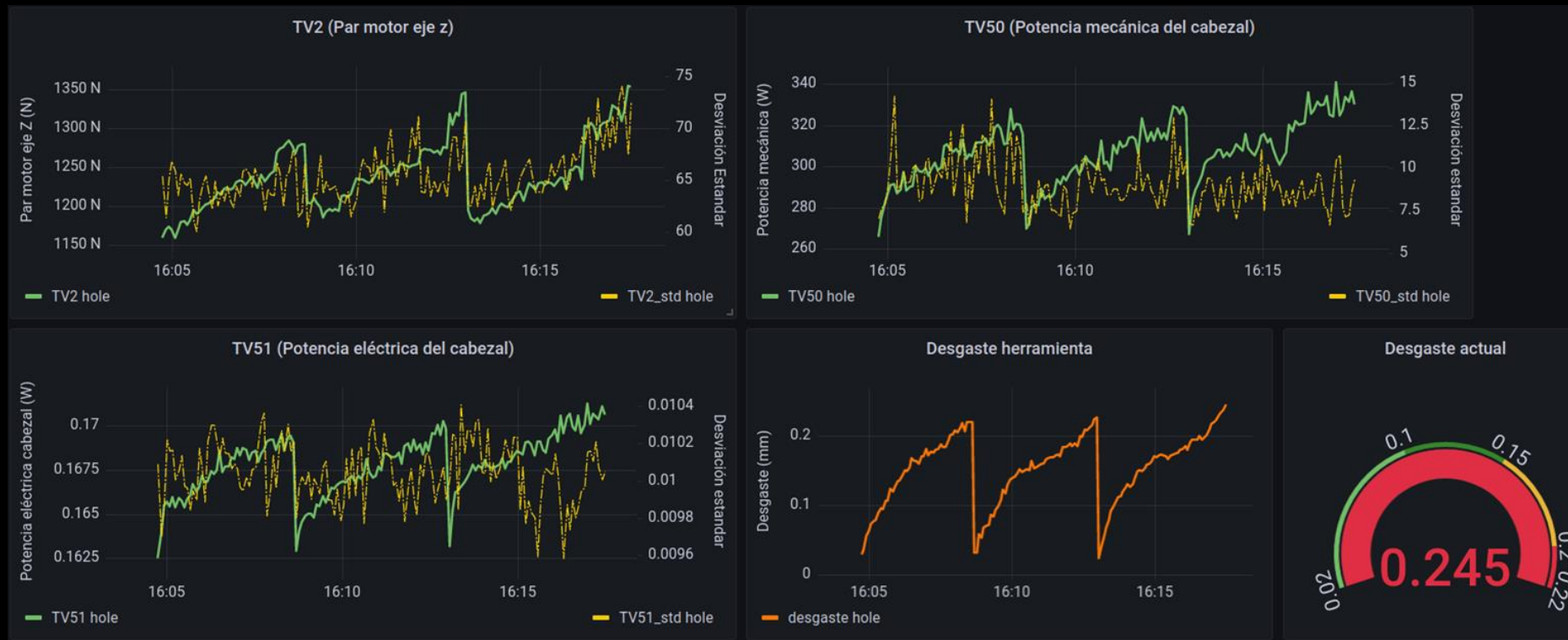


The image shows a screenshot of the Grafana interface's time range selector. It consists of two input fields: 'From' and 'To'. The 'From' field contains the text 'now-15m' and has a calendar icon to its right. The 'To' field contains the text 'now' and also has a calendar icon to its right. To the right of these fields is a dropdown menu that is currently open, displaying three options: 'Last 5 minutes', 'Last 15 minutes', and 'Last 30 minutes'. The 'Last 15 minutes' option is highlighted with a red rectangular border, indicating it is the selected time range.

- Como antes, decimos que se actualicen las gráficas cada 5 segundos.

MOSTRAR DATOS EN TIEMPO REAL (STREAMING)

- Irán apareciendo las gráficas y actualizándose.



GRAFANA EJEMPLOS

- <https://play.grafana.org/d/000000012/grafana-play-home?orgId=1>